Depression Detection from Reddit Posts

K. Bhanu Prakash
CB.SC.U4AIE23134

M. Sravanthi Suma
CB.SC.U4AIE23148

Nandana Thachilath M.
CB.SC.U4AIE23151

Y. Navaddip Chowdary
CB.SC.U4AIE23163

***Abstract*—Depression is one of the most prevalent mental health disorders worldwide, and early detection is critical for timely intervention. Social media platforms such as Reddit host millions of posts daily from individuals discussing mental health, offering an unprecedented opportunity for large-scale automated depression detection. This paper presents a deep learning approach for binary classification of Reddit posts as *Depressed* or *Not Depressed* using a fine-tuned BERT (Bidirectional Encoder Representations from Transformers) model combined with FastText word embeddings. We collected and preprocessed a dataset of 831,138 Reddit posts from five subreddits, applying a complete NLP pipeline including noise removal, tokenization, stopword filtering, stemming, and lemmatization. Our fine-tuned BERT model achieved an accuracy of 91.59%, F1-Score of 90.65, AUC-ROC of 0.9535, and a Recall of 91.28% on the test set. We also employ LIME (Local Interpretable Model-agnostic Explanations) to interpret the model's predictions. A comparative evaluation shows BERT outperforms a BiLSTM baseline across all metrics, demonstrating the efficacy of transformer-based models for mental health text classification.**

***Index Terms*—Depression Detection, BERT, FastText, Reddit, NLP, Deep Learning, Mental Health, Binary Classification, Transformer, LIME**

I. Introduction

Mental health disorders, particularly depression, represent a growing global public health crisis. According to the World Health Organization (WHO), over 280 million people suffer from depression worldwide. Despite its prevalence, depression remains significantly underdiagnosed due to the social stigma attached to mental illness and the difficulty individuals face in openly seeking help.

Social media platforms, especially Reddit, have emerged as important venues where people openly discuss their mental health. Communities such as `r/depression` and `r/SuicideWatch` attract millions of posts from individuals expressing hopelessness, feelings of worthlessness, and suicidal ideation. These platforms offer an unprecedented opportunity to detect and flag posts that may indicate clinical depression, enabling early intervention by mental health professionals.

However, the scale of social media data makes manual analysis infeasible. Reddit generates millions of posts daily, and no human team can read and flag them all. This motivates the development of automated, AI-powered solutions capable of scanning text at scale and detecting linguistic patterns associated with depression.

This paper makes the following contributions:

- A comprehensive NLP pipeline for preprocessing Reddit posts at scale (831,138 posts).
- Training a FastText model to generate 300-dimensional word embeddings capturing semantic meaning.
- Fine-tuning a BERT (`bert-base-uncased`) model for binary depression classification with advanced hyperparameter configuration.
- Comparative analysis of BERT vs. BiLSTM models.
- Interpretability analysis using LIME to explain model predictions.

II. Related Work

Several studies have explored computational approaches to mental health detection from social media. Early work by [8] used linguistic features and machine learning classifiers (SVM, Naive Bayes) on Twitter data to detect depression. Subsequent work leveraged LIWC (Linguistic Inquiry and Word Count) features combined with sentiment analysis.

More recent approaches have adopted deep learning. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks [6] demonstrated improved performance by capturing sequential dependencies in text. Bidirectional LSTMs (BiLSTMs) further improved results by processing text in both forward and backward directions [7].

The introduction of transformer-based models, particularly BERT [1], revolutionized NLP by enabling bidirectional context representation through self-attention mechanisms [2]. Fine-tuning BERT for downstream classification tasks has consistently outperformed RNN-based approaches across a wide range of text classification benchmarks.

FastText [3], developed by Facebook AI Research, generates word embeddings by considering subword n-grams, making it particularly effective for handling out-of-vocabulary words and misspellings common in social media text. Prior work by [5] on Word2Vec laid the foundation for dense word representations used in NLP pipelines. Interpretability of deep learning models has been addressed through tools such as LIME [10] and SHAP [11], which highlight influential features driving model predictions.

III. Dataset

A. Data Collection

We collected posts from five Reddit communities (subreddits) using the Reddit API. Posts were labeled into two classes based on the subreddit of origin:

Depressed (Label = 1):

- `r/depression` – Posts about feeling hopeless, empty, and unable to cope with daily life.

- `r/SuicideWatch` – Posts about suicidal thoughts, feeling like a burden, and wanting pain to stop.

Not Depressed (Label = 0):

- `r/teenagers` – School life, gaming, friendships; normal everyday teenage content.
- `r/Happy` – Positive experiences, gratitude, joyful moments, and good news.
- `r/DeepThoughts` – Philosophical curiosity and neutral reflection (not emotional distress).

Key vocabulary indicators in depressed posts include words such as *"hopeless, worthless, empty, burden, suicidal, darkness"*, while non-depressed posts are characterized by terms like *"excited, happy, grateful, fun, gaming, blessed"*. Similar subreddit-based labeling strategies have been employed in prior mental health NLP studies [9].

B. Dataset Statistics

The dataset was balanced to avoid class imbalance bias, resulting in equal representation of both classes. The final dataset statistics are shown in Table I.

TABLE I
DATASET SPLIT STATISTICS

Split	Posts	Percentage
Training	664,993	80%
Validation	83,031	10%
Test	83,114	10%
Total	**831,138**	**100%**

The test set contained 42,513 Non-Depressed samples and 40,601 Depressed samples, confirming near-balanced class distribution.

IV. METHODOLOGY

Our system pipeline consists of four major steps: data collection and labeling, text preprocessing, feature extraction via word embeddings, and classification using fine-tuned BERT.

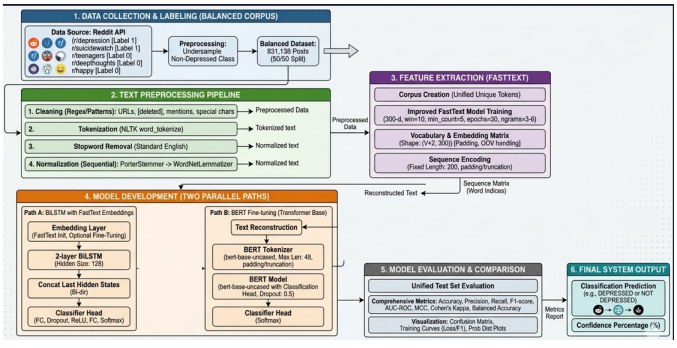

Fig. 1. Full System Architecture: From Reddit data collection through preprocessing, FastText feature extraction, and BERT classification to final output.

A. Step 1: Text Preprocessing Pipeline

Raw Reddit posts contain significant noise including URLs, special characters, Reddit-specific patterns (e.g., `u/username`, `r/subreddit`), and profanity. We applied the following preprocessing steps sequentially:

1) **Lowercasing:** Convert all text to lowercase to ensure uniformity.
2) **Noise Removal:** Remove URLs, special characters, and Reddit-specific patterns using regular expressions.
3) **Tokenization:** Split text into individual word tokens using NLTK's `word_tokenize`.
4) **Stopword Removal:** Remove common stopwords (e.g., *the, and, is, so, me, like*) that carry minimal semantic meaning.
5) **Stemming + Lemmatization:** Apply Porter Stemmer and WordNet Lemmatizer to reduce words to their base forms (e.g., *going* $\rightarrow$ *go*, *hopeless* $\rightarrow$ *hopeless*).

Example: Consider the input sentence:

> *"I feel so empty and hopeless, nothing brings me joy anymore. I can't keep going like this."*

After the full preprocessing pipeline, the 17-token sentence is reduced to 10 normalized tokens: `['feel', 'empti', 'hopeless', 'noth', 'bring', 'joy', 'anymor', 'cant', 'keep', 'go']`.

B. Step 2: FastText Word Embeddings

We trained a FastText model on the cleaned dataset to generate 300-dimensional dense word vectors. FastText's subword n-gram approach makes it robust to out-of-vocabulary words and spelling variations common in social media text [4].

Each word w is mapped to a vector $\mathbf{v}_w \in \mathbb{R}^{300}$ via:

$$\mathbf{v}_w = \frac{1}{|\mathcal{G}_w|} \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \tag{1}$$

where $\mathcal{G}_w$ is the set of character n-grams for word w and $\mathbf{z}_g$ is the embedding vector for n-gram g.

The resulting embedding matrix has dimensions vocab_size $\times$ 300 (e.g., $15{,}000 \times 300 = 4{,}500{,}000$ parameters). For the FastText path, token sequences are padded/truncated to a fixed length of 200 tokens:

$$\mathbf{S} = [w_1, w_2, \ldots, w_{200}] \in \mathbb{Z}^{200} \tag{2}$$

C. Step 3: BERT Tokenization and Encoding

For the BERT path, preprocessed text is re-tokenized using BERT's WordPiece tokenizer (`BertTokenizer`) with a maximum sequence length of 48 tokens. The tokenizer produces:

- **Token IDs** (`input_ids`): Integer IDs for each subword token.
- **Attention Mask**: Binary mask indicating real tokens (1) vs. padding (0).
- **Segment Embeddings**: All zeros for single-sentence classification.

The BERT input representation combines token embeddings, position embeddings, and segment embeddings:

$$\mathbf{E}_{input} = \mathbf{E}_{token} + \mathbf{E}_{position} + \mathbf{E}_{segment} \tag{3}$$

BERT internal dimensions for `bert-base-uncased`:

- Token Embeddings: $(1, 48, 768)$ – each of 48 tokens is a 768-dim vector.
- Transformer Layer $\times 12$: $(1, 48, 768)$ – 12 self-attention heads $\times$ 64 dimensions.
- [CLS] Token Vector: $(1, 768)$ – sentence representation.
- Classifier Head: $(1, 2)$ – linear layer $768 \rightarrow 2$ logits.

D. Step 4: BERT Fine-Tuning Architecture

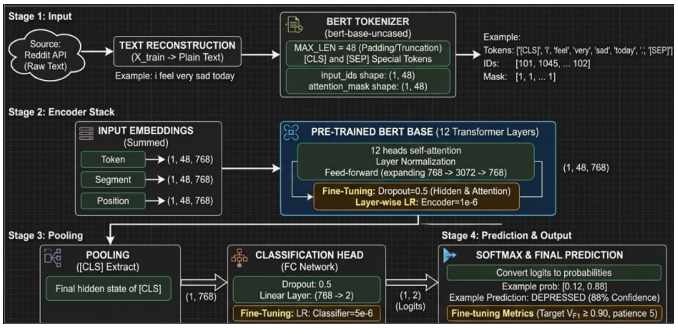

Fig. 2. BERT Internal Working Architecture: Tokenization, 12 transformer layers, task-specific classification head, and backpropagation with weight updates.

The `bert-base-uncased` model consists of 12 transformer encoder layers, each comprising Multi-Head Self-Attention, Feed-Forward Networks, Layer Normalization, and Residual Connections [2].

The self-attention mechanism computes:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{4}$$

where $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ are the query, key, and value matrices, and $d_k = 64$.

Multi-head attention with $h = 12$ heads:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{5}$$

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q,\ \mathbf{K}\mathbf{W}_i^K,\ \mathbf{V}\mathbf{W}_i^V) \tag{6}$$

The **Task-Specific Classification Head** attached to the [CLS] vector consists of: Dense Layer (768 units) $\rightarrow$ Dropout (rate = 0.5) $\rightarrow$ Linear Layer ($768 \rightarrow 2$) $\rightarrow$ Softmax.

The final prediction probability for class c is:

$$P(y = c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^{2} e^{z_j}} \tag{7}$$

where z_c is the logit for class c.

V. BERT Fine-Tuning Hyperparameters

A. Model Architecture

TABLE II
BERT Model Architecture Configuration

Parameter	Value
Base Model	bert-base-uncased
Classification Type	Binary
Number of Classes	2
Dropout Rate	0.5
Gradient Checkpointing	Enabled

B. Input & Tokenization

TABLE III
Tokenization Configuration

Parameter	Value
Tokenizer	BertTokenizer (WordPiece)
Maximum Sequence Length	48 tokens
Padding Strategy	max_length
Truncation	Enabled

C. Training Configuration

TABLE IV
Training Hyperparameters

Parameter	Value
Epochs	30
Batch Size	32 (fallback: 24)
Gradient Accumulation Steps	4
Effective Batch Size	128
DataLoader Workers	4
Random Seed	42

D. Optimizer

We employed the AdamW optimizer [12] with layer-wise learning rates:

$$\theta_{t+1} = \theta_t - \alpha_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta_t\right) \tag{8}$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected first and second moment estimates, and $\lambda = 0.01$ is the weight decay coefficient.

TABLE V
AdamW Optimizer Configuration

Parameter	Value
Optimizer	AdamW
Encoder Learning Rate	1×10^{-6}
Classifier Learning Rate	5×10^{-6}
Weight Decay (λ)	0.01

E. Learning Rate Scheduler

We used OneCycleLR with cosine annealing:

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})\left(1 + \cos\left(\frac{\pi \cdot t}{T}\right)\right) \quad (9)$$

TABLE VI
ONECYCLELR SCHEDULER CONFIGURATION

Parameter	Value
Scheduler	OneCycleLR
Total Steps	(len(train_loader) × epochs)/grad_accum
Warm-up Percentage	10%
Annealing Strategy	Cosine

F. Loss Function

We employed Cross-Entropy Loss with class weighting:

$$\mathcal{L} = -\sum_{c=1}^{C} w_c \cdot y_c \log(\hat{p}_c) \quad (10)$$

where w_c is the class weight computed using `compute_class_weight()`, y_c is the ground truth indicator, and $\hat{p}_c$ is the predicted probability.

G. Regularization Techniques

TABLE VII
REGULARIZATION CONFIGURATION

Technique	Value
Dropout	0.5
Gradient Clipping	1.0 (max norm)
Weight Decay	0.01

Gradient clipping prevents exploding gradients:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{\delta}{\max(\|\mathbf{g}\|, \delta)}, \quad \delta = 1.0 \quad (11)$$

H. Early Stopping

TABLE VIII
EARLY STOPPING CONFIGURATION

Parameter	Value
Patience	5 epochs
Minimum Delta	0.0005
Target Validation F1	0.90
Stop Condition	F1 $\geq$ 0.90

I. Mixed Precision & Hardware Optimizations

Automatic Mixed Precision (AMP) was enabled using PyTorch's `autocast` and `GradScaler` to reduce GPU memory usage and accelerate training. Additional hardware optimizations: TF32 MatMul, CuDNN Benchmark, Gradient Checkpointing, and Pin Memory — all enabled.

J. Model Quantization

Post-training Dynamic INT8 Quantization was applied to all linear layers:

$$\mathbf{W}_{int8} = \text{round}\left(\frac{\mathbf{W}_{fp32}}{s}\right), \quad s = \frac{\max(|\mathbf{W}_{fp32}|)}{127} \quad (12)$$

This reduces model size and accelerates inference with negligible accuracy loss.

VI. RESULTS

A. Performance Metrics

The following evaluation metrics were computed on the held-out test set of 83,114 samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (13)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \quad (14)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (15)$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (16)$$

TABLE IX
BERT MODEL PERFORMANCE ON TEST SET (83,114 SAMPLES)

Metric	Value
Accuracy	91.59%
Precision	0.8618
Recall	0.9128
F1-Score	90.65
AUC-ROC	0.9535
MCC	0.7732

B. Confusion Matrix

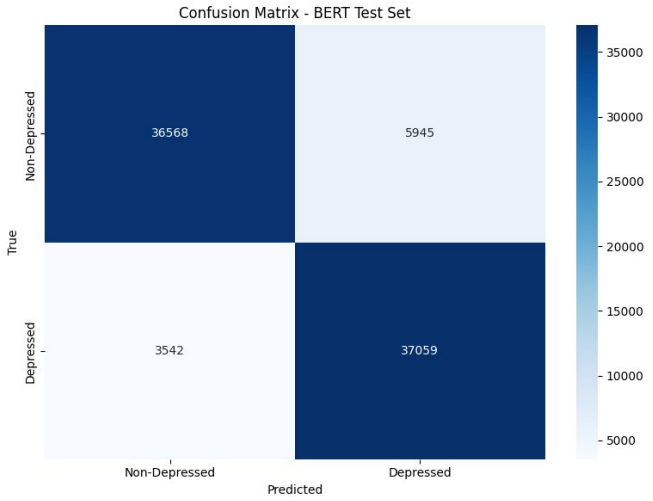

Fig. 3. Confusion Matrix on the BERT Test Set (83,114 samples).

TABLE X
CONFUSION MATRIX VALUES

		Predicted	
		Non-Dep.	**Depressed**
Actual	Non-Dep.	36,568 (TN)	5,945 (FP)
	Depressed	3,542 (FN)	37,059 (TP)

- **TP = 37,059:** Model correctly predicted Depressed.
- **TN = 36,568:** Model correctly predicted Non-Depressed.
- **FP = 5,945:** Model incorrectly predicted Depressed (actually Normal).
- **FN = 3,542:** Model incorrectly predicted Non-Depressed (actually Depressed).

The low False Negative rate (3,542) is particularly important for the clinical use case, as missing a truly depressed individual carries a higher cost than a false alarm.

VII. MODEL COMPARISON: BERT VS. BILSTM

TABLE XI
BILSTM VS. BERT: TEST SET PERFORMANCE COMPARISON

Metric	**BiLSTM**	**BERT**
Accuracy	87.19%	**91.59%** (↑+4.40%)
Precision	84.78%	**86.18%** (↑+1.40%)
Recall	89.92%	**91.28%** (↑+1.36%)
F1-Score	87.27%	**90.65%** (↑+3.38%)
AUC-ROC	94.59%	**95.35%** (↑+0.76%)
MCC	0.7453	**0.7732** (↑+0.028)

BERT outperforms BiLSTM across all evaluation metrics. The most significant gains are in Accuracy (+4.40%) and F1-Score (+3.38%). This improvement is attributed to BERT's bidirectional context modeling via self-attention, pre-trained language understanding, and its ability to capture long-range dependencies in text [13].

VIII. EXPLAINABILITY WITH LIME

To improve transparency and trust in the model's predictions, we applied LIME (Local Interpretable Model-agnostic Explanations) [10]. LIME explains individual predictions by fitting a locally interpretable linear model around each input sample.

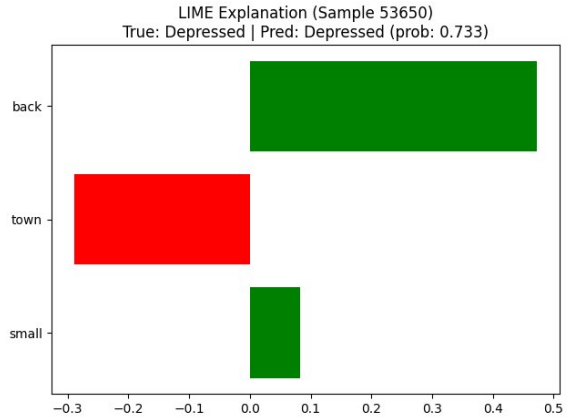

Fig. 4. LIME Explanation for Sample #53650 (True: Depressed, Predicted: Depressed, Prob: 0.733). Green bars support the Depressed prediction; red bars oppose it.

For Sample #53650, the word contributions are:

- **"back"** → +0.4718 (strongly supports Depressed)
- **"small"** → +0.0835 (weakly supports Depressed)
- **"town"** → -0.2889 (supports Non-Depressed)

LIME makes the AI model transparent and trustworthy by revealing which words in a post most influence the classification decision [11].

IX. DISCUSSION

The results demonstrate that fine-tuned BERT achieves strong performance (91.59% accuracy, 95.35% AUC-ROC) on depression detection from Reddit posts. Several key observations:

Strength of BERT: BERT's bidirectional context modeling allows it to understand nuanced expressions of depression such as metaphorical language, indirect references to hopelessness, and contextual sentiment shifts that simpler models miss [14].

FastText Complementarity: FastText embeddings serve as a complementary representation, particularly useful for handling out-of-vocabulary words, typos, and abbreviations common in Reddit text [4].

False Negatives: The 3,542 false negatives represent the critical failure mode in a mental health application. Future work should prioritize reducing FN by adjusting the classification threshold [15].

Limitations: The model cannot replace clinical diagnosis. It serves as a screening tool to flag posts for human review. Privacy concerns and ethical considerations around automated monitoring of user posts must be carefully addressed [16].

X. CONCLUSION

This paper presented a BERT + FastText-based system for detecting depression from Reddit posts. Our system achieved 91.59% accuracy and 95.35% AUC-ROC on an 831,138-post dataset, outperforming a BiLSTM baseline by 4.40% in accuracy. The system employs a complete NLP pipeline, advanced BERT fine-tuning with layer-wise learning rates, OneCycleLR scheduling, mixed precision training, and LIME-based explainability.

The results demonstrate the viability of transformer-based NLP models for mental health text classification at scale. Future work will explore multi-class severity detection, incorporation of post metadata, ensemble methods, and clinical validation with mental health professionals.

ACKNOWLEDGMENT

The authors thank their institution for providing computational resources and guidance. This work was conducted as part of the NLP & Deep Learning coursework (Group B-17).

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[3] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proc. 15th Conf. European Chapter of the ACL*, 2017, pp. 427–431.

[4] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.

[5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 3111–3119.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[8] G. Coppersmith, M. Dredze, and C. Harman, "Quantifying mental health signals in Twitter," in *Proc. ACL Workshop on Computational Linguistics and Clinical Psychology*, 2014, pp. 51–60.

[9] A. Yates, A. Cohan, and N. Goharian, "Depression and self-harm risk assessment in online forums," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 2968–2978.

[10] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4765–4774.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th Int. Conf. on Learning Representations (ICLR)*, 2019.

[13] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune BERT for text classification?," in *Proc. China National Conf. on Chinese Computational Linguistics (CCL)*, 2019, pp. 194–206.

[14] S. Ji, S. Pan, E. Cambria, P. Marttinen, and P. S. Yu, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 494–514, 2022.

[15] S. Chancellor and M. D. Choudhury, "Methods in predictive techniques for mental health status on social media: A critical review," *npj Digital Medicine*, vol. 3, no. 1, p. 43, 2020.

[16] A. Benton, M. Mitchell, and D. Hovy, "Multi-task learning for mental health using social media text," in *Proc. 15th Conf. of the European Chapter of the ACL (EACL)*, 2017, pp. 152–162.